Summary Notes:
Infrastructure as Code (IaC) on Azure

Overview

Here is an overview of the topics covered in this session.

1. Session 1
    a. **Git and GitHub Integration**
    b. **Git Workflow with Azure DevOps**
    c. **Managing Build Dependencies**

2. Session 2
    a. **Build Agents in Azure Pipelines**
    b. **Azure Pipelines**
    c. **Deployment Patterns**
    d. **Enhancing Pipeline Quality**

Session 1

Git and GitHub Integration

- **Traditional CVCS Challenges**: Single-point failures and limited feature experimentation.
- **Git Benefits**:
    - Distributed version control with independent repositories.
    - Supports collaboration via feature branching and pull requests.

Git Workflow With Azure DevOps

1. Clone repository: Sync local repository with the main branch.
2. Feature branching: Develop features in separate branches.
3. Pull requests: Submit changes for review and approval.
4. Merge: Integrate approved changes into the main branch.

Managing Build Dependencies

- **Importance**: Ensures code consistency, prevents drift, and centralizes updates.
- **Efficient Management**:
    - Identify reusable components (e.g., Maven, NPM, NuGet packages).
    - Host feeds via Azure Artifacts (public/private).
    - Use secure tokens for authentication.

Key Tools

- **Azure Artifacts**: Integrates artifact usage with workflows.
- **Version Control**: Tracks updates using semantic versioning.

Semantic Versioning:

- **Format**: Major.Minor.Patch (e.g., 2.5.1)
    - Major: Breaking changes.
    - Minor: Backward-compatible features.
    - Patch: Backward-compatible fixes.

Azure Artifacts Views:

- @local: In-development packages.
- @pre-release: For testing.
- @release: Ready for production.

Session 2

Build Agents in Azure Pipelines

- **Microsoft-Hosted Agents**: Pre-installed tools, less customizable.
- **Private Agents**: Self-hosted for custom builds and resources.

Setup Methods:

- Manual configuration, scripted setup, or pre-configured VM templates.

Azure Pipelines

- **Purpose**:
    - Automate Continuous Integration (CI) and Continuous Delivery (CD).
    - Build, test, and deploy with each code commit.

Pipeline Stages:

1. **Build**: Compile and generate artifacts.
2. **Test**: Run unit, integration, and performance tests.
3. **Deploy**: Deliver artifacts to environments.

Triggers:

- CI, PR, scheduled, and build completion.
- Conditional logic for dynamic execution.

Deployment Patterns

- **Blue-Green Deployment**: Minimizes downtime using two environments.
    1. Deploy to staging (Green)
    2. Validate and swap with live (Blue)

Enhancing Pipeline Quality

- **Automated Testing**: Unit, integration, and load tests.
- **Monitoring & Alerts**:
    - Use Azure Monitor for build/test metrics.
    - Set alerts for failed builds or delays.